

The background features a complex network of thin, light-colored lines and dots, forming various triangular shapes and a larger, more intricate web-like structure on the left side. The overall aesthetic is technical and modern.

I2C Driver Overview

Introduction to the Custom Interrupt-Driven I2C Driver with Semaphore-Based Synchronization

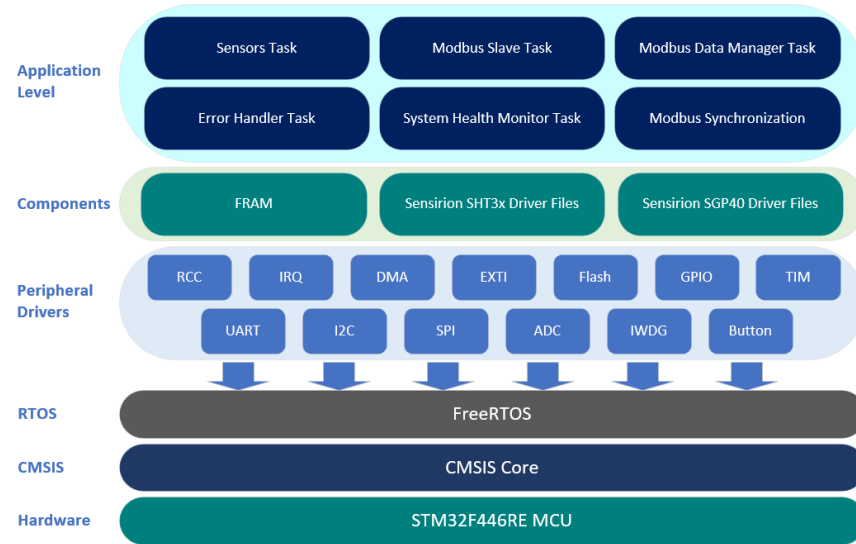
I2C Driver Overview

- I2C Driver Overview
- I2C Driver Synchronization Overview
- I2C Driver Synchronization – How it Works
- I2C Driver Synchronization – Example
- I2C Driver Synchronization – Example...
- I2C Driver Source Code Review & Integration

I2C Driver Overview

General

- **Purpose & Functionality:** Implements a custom interrupt-driven I2C driver for initializing I2C instances and performing master transmit/receive operations, using FreeRTOS semaphores for synchronization, ensuring efficient communication.
- **Project Significance:**
 - **Environmental Sensors:** Enables communication with environmental sensors (SHT3x & SGP40).
 - **Real-Time System Compatibility:** The driver is optimized for real-time system performance and reliability, utilizing FreeRTOS synchronization methods.



I2C Driver Synchronization Overview

- **Purpose of Synchronization**

- In an interrupt-driven system, synchronization is essential for managing data flow and ensuring the I2C communication events (like transmitting or receiving data) occur in the right sequence.

- **Why Semaphores & Interrupts?**

- **Interrupts** allow the I2C driver to respond to hardware events (like data transmission complete or buffer full) without constantly polling the status, which is inefficient.
- **Semaphores** are used to wait for these specific events. The driver "takes" a semaphore while waiting for a specific interrupt, and the interrupt handler "gives" the semaphore when the event occurs, signaling the driver to proceed.

- **Benefits & Trade-offs**

- **Benefits:** Efficient handling of I2C events without wasting CPU cycles on polling, ensures accurate timing, and allows for real-time responsiveness.
- **Trade-offs:** Slightly more complex to implement due to managing multiple interrupts (especially with I2C) and synchronization states.

I2C Driver Synchronization – How it Works

Synchronization Methods: I2C Master Transmit and Receive functions utilize synchronization functions which handle enabling an interrupt, then “take” a semaphore while waiting for a specific interrupt, and the interrupt handler “gives” the semaphore when the event occurs, signaling the driver to proceed.

- `i2c_synch_txe_interrupt`: Waits for the transmit buffer to be empty, synchronizing the transmission process.
- `i2c_synch_rxne_interrupt`: Waits for data to be received in the receive buffer, synchronizing the data reception.
- `i2c_synch_btfe_interrupt`: Handles the Byte Transfer Finished flag, signaling when all bytes have been transferred.

```
static error_t i2c_synch_txe_interrupt(I2C_TypeDef *i2c_instance)
{
    error_t status = ERR_OK;

    // Set the TXE synch flag and enable interrupt
    i2c_synch_flags.I2C1_WAIT_TXE = true;
    i2c_enable_rx_tx_interrupt(i2c_instance);

    // Wait until TXE is set
    if (xSemaphoreTake(i2c_get_semaphore_handle(i2c_instance), I2C_TIMEOUT_TICKS) != pdTRUE)
    {
        status = ERR_TIMEOUT;
    }

    return status;
}
```

I2C Driver Synchronization – Example

I2C Master Transmit Function

Transmits data from the STM32 to an I2C slave using interrupts and synchronization mechanisms.

- **i2c_start_transmit:** Begins the transmission by sending the Start Condition, followed by the Slave Address, and are synchronized by using the SB (Start Bit) and ADDR interrupts.
- **i2c_synch_txe_interrupt:** The TXE (Transmit Data Register Empty) interrupt ensures that the buffer is ready for the next byte. The semaphore is taken until the interrupt is triggered, indicating the buffer is ready.
- **i2c_synch_btf_interrupt:** Waits for the BTF (Byte Transfer Finished) flag, which indicates that the previous byte has been transmitted.
- **Interrupts:** Each synchronization function is driven by specific interrupts that "give" semaphores.

```
error_t i2c_master_transmit(I2C_TypeDef *i2c_instance, uint8_t device_address, uint8_t *tx_buffer, uint16_t data_length)
{
    error_t status = ERR_OK;

    if (i2c_wait_for_busy_clear(i2c_instance) == ERR_OK)
    {
        if (i2c_start_transmit(i2c_instance, device_address) == ERR_OK)
        {
            uint16_t tx_count = data_length;
            uint16_t tx_byte_num = 0;

            // Send the data
            while (tx_count)
            {
                // Wait for TXE
                if (i2c_synch_txe_interrupt(i2c_instance) != ERR_OK)
                {
                    status = ERR_TIMEOUT;
                    break;
                }

                // Send the byte
                *(__IO uint8_t *)&i2c_instance->DR = tx_buffer[tx_byte_num++];

                tx_count--;

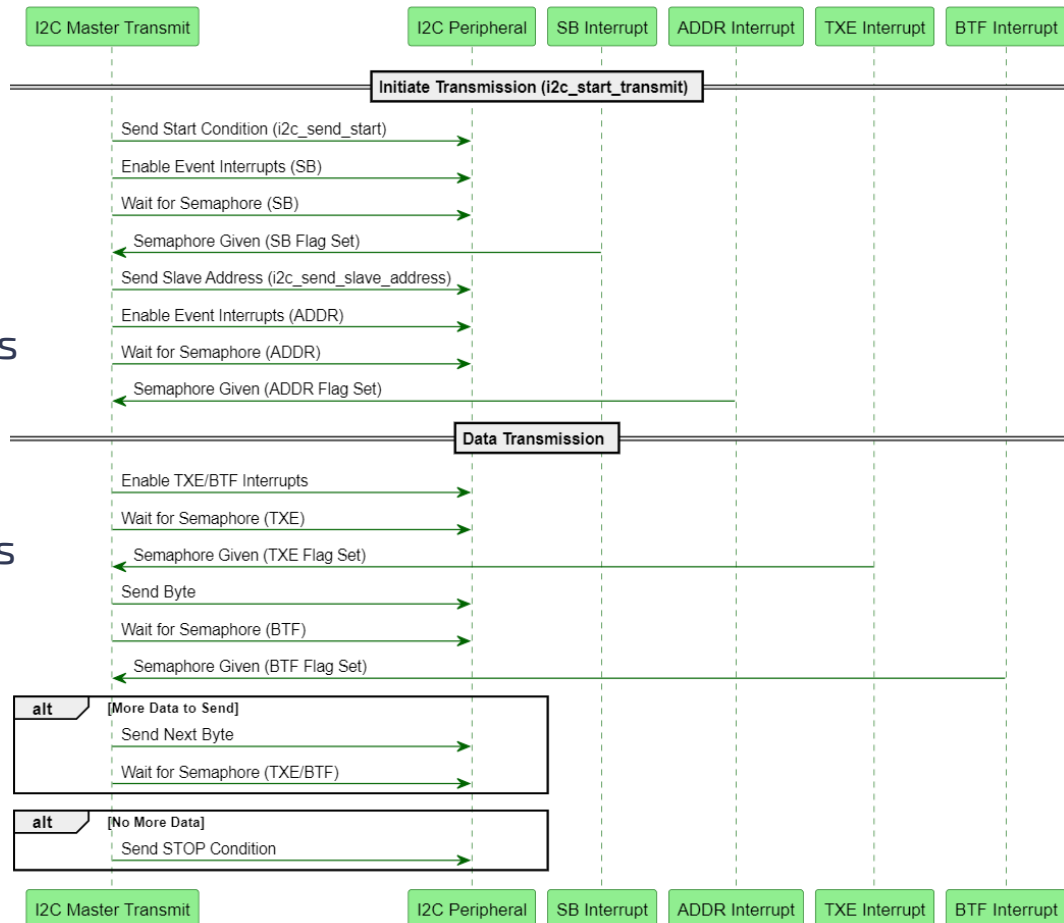
                // Wait for BTF
                if (i2c_synch_btf_interrupt(i2c_instance) != ERR_OK)
                {
                    status = ERR_TIMEOUT;
                    break;
                }
            }

            // Send STOP condition
            i2c_instance->CR1 |= (I2C_CR1_STOP);
        }
        else
        {
            status = ERR_FAIL;
        }
    }
    else
    {
        status = ERR_FAIL;
    }

    return status;
}
```

I2C Driver Synchronization – Example...

- **Transmit Initialization:** Initiates the **I2C master transmit** process by sending the start condition and slave address.
- **Synchronization with Semaphores:** Sets synchronization flags and enables interrupts for Start Bit (SB), Address (ADDR), Transmit Buffer Empty (TXE), and Byte Transfer Finished (BTF) events, each handled with semaphores for precise timing control.
- **Data Transmission:** Sends bytes of data one by one, with synchronization handled for each byte using TXE and BTF interrupts to ensure data is transmitted correctly and fully.



I2C Driver Source Code Review & Integration

Overview

- **Objective:** Review and integrate the custom interrupt-driven I2C driver, focusing on how semaphore synchronization is used to manage communication in an RTOS-based system.
- **Key Steps:**
 - **Understand:** The structure and design of the custom I2C driver.
 - **Integrate and Review:** Header file definitions, structures, function prototypes, and source file functions, including interrupt handlers.
- **Project Significance:**
 - Gain a comprehensive understanding of how the I2C driver leverages semaphores and interrupts to synchronize operations, ensuring efficient communication in a real-time system.

The background features a complex network of thin, light-colored lines and small circles, forming various triangular shapes. Some of these shapes are filled with a very light yellow or green color. The overall effect is a subtle, geometric pattern that resembles a wireframe or a network diagram.

Next: I2C Header File
